the current production system, if one exists. Many teams even record a selection of production data for use in load testing. If you're building the entire system from scratch, on the other hand, the team would have to agree on some figures based on the expected usage patterns.

Cucumber, Java, and The Grinder for real-world performance testing

When Tom Howard needed to verify the performance of an application built around a messaging platform for a large electricity company, he chose to use a BDD approach.[5] To do this, he used The Grinder and Cucumber-JVM to automate a BDD performance scenario similar to the one discussed here. The main Cucumber step invoked a Grinder script directly and waited for it to finish. The Grinder script ran a series of requests based on real production data from a number of worker machines to simulate production-like loads.

Once the script was done, the test results were recorded in a CSV log file. Tom used open source libraries to parse the CSV file and determine the 95th and 100th percentiles, which were in turn used to decide whether the scenario passed or failed.

It might be tempting to write automated tests for more low-level performance metrics such as memory consumption, CPU usage, average response time, and so forth. But this is generally not a cost-effective approach. During performance testing, a system can underperform or fail for many different reasons: insufficient memory, a slow database, network issues, suboptimal code, and so forth. In addition, in the real world, performance issues inevitably come from unexpected places, and it's very hard to predict or test for every possible problem.

But in business terms, poor performance at any point will only really be of concern if it's reflected in the performance requirements described in the BDD scenario, and this will be reflected by a failing BDD scenario.

A more effective approach is to monitor and record low-level performance metrics such as memory and CPU usage, but not to include these in any automated tests. If the BDD scenario does fail, then you've identified a performance issue that can potentially impact the business. In this case, you can use the recorded metrics to investigate and troubleshoot the issue.

9.5 Discovering the design

BDD is an excellent way to design clean, reusable service APIs, as well as clean, reusable APIs at all levels of the application. Well-designed applications are typically organized into layers and components (in design patterns, this principle is often referred to as the *separation of concerns*). If you think of an application in terms of these layers and components, starting from the UI and working down, each layer is the "user" of the next layer down, and any code that uses a component is effectively a user of that component. When the

[5] Tom Howard, "Cucumber-JVM + The Grinder + Math = Performance.Testing.Heaven," http://windyroad.com.au/2013/09/11/cucumber-jvm-the-grinder-math-performance-testing-heaven.